CarLog — Frontend

JaviRSDEV

[Angular v21] [Tailwind CSS 4.1]

[TypeScript 5.9] [Vitest 4.0]

[Bootstrap 5.3] [SonarQube A]

CarLog es una aplicación web que digitaliza el ciclo de vida completo del mantenimiento vehicular: desde el alta del taller hasta la facturación detallada por órdenes de trabajo. Ofrece interfaces especializadas para tres roles de usuario: **Gerentes**, **Mecánicos** y **Clientes**.

Índice

1. Estado del proyecto **3**

2. Tech Stack **3**

3. Requisitos previos **3**

4. Instalación **3**

5. Configuración del entorno **4**
5.1. Desarrollo local 4
5.2. Producción 4
5.3. Configuración WebSocket 4

6. Scripts disponibles **4**

7. Arquitectura **5**

8. Rutas y navegación **5**

9. Seguridad y autenticación **6**
9.1. Persistencia de sesión 6
9.2. Flujo de autenticación 6

10. Interceptores HTTP **7**
10.1. `authInterceptor` 7
10.2. `ErrorInterceptor` 7

11. Funcionalidades por rol **7**

12. Notificaciones en tiempo real **7**
12.1. Tipos de notificación 7

13. Conexión con el Backend **8**

14. Licencia **8**

1. Estado del proyecto

- [Quality Gate Status]
- [Maintainability Rating]
- [Security Rating]
- [Reliability Rating]
- [Duplicated Lines (%)]

2. Tech Stack

Tecnología	Versión	Uso
Angular	21.2	Framework principal (Standalone Components)
TypeScript	5.9	Lenguaje de desarrollo
RxJS	7.8	Programación reactiva
Bootstrap	5.3	Sistema de grid y componentes UI
Bootstrap Icons	1.13	Iconografía
Tailwind CSS	4.1	Utilidades CSS adicionales
@ng-bootstrap	20.0	Componentes Bootstrap para Angular
SweetAlert2	11	Alertas y modales de usuario
@stomp/rx-stomp	1.2	Cliente WebSocket con protocolo STOMP
SockJS	1.6	Fallback WebSocket
ngx-cookie-service	21.1	Gestión de cookies
Vitest	4.0+	Testing unitario
Prettier	3.8	Formateo de código
Node.js	20+	Entorno de ejecución
npm	11+	Gestor de paquetes

3. Requisitos previos

- Node.js v20 o superior
- npm v11 o superior
- Backend Spring Boot corriendo (por defecto en `http://localhost:8081`)

4. Instalación

```
# 1. Clonar el repositorio
git clone https://github.com/JaviRSDEV/FrontCarLog.git
cd FrontCarLog
```

```
# 2. Instalar dependencias
npm install
```

5. Configuración del entorno

Los ficheros de entorno **no están incluidos en el repositorio** (están en `.gitignore`). Deben crearse manualmente.

5.1. Desarrollo local

Crea `src/environments/environment.development.ts`:

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8081',
  wsUrl: 'ws://localhost:8081/ws-carlog',
};
```

5.2. Producción

Crea `src/environments/environment.ts`:

```
export const environment = {
  production: true,
  apiUrl: 'https://tu-api-en-produccion.com',
  wsUrl: 'wss://tu-api-en-produccion.com/ws-carlog',
};
```

5.3. Configuración WebSocket

La conexión WebSocket se configura en `src/app/config/rx-stomp-config.ts`:

```
export const myRxStompConfig: RxStompConfig = {
  brokerURL: environment.wsUrl,
  heartbeatIncoming: 0,
  heartbeatOutgoing: 20000,
  reconnectDelay: 200,
};
```

6. Scripts disponibles

Comando	Descripción
`npm start`	Inicia el servidor de desarrollo en el puerto 4200

Comando	Descripción
`npm run build`	Compila el proyecto para producción en `dist/`
`npm run watch`	Compila en modo watch (recarga automática)
`npm test`	Ejecuta los tests unitarios con Vitest

7. Arquitectura

CarLog está construido con **Angular 21** siguiendo la arquitectura de **Standalone Components** (sin NgModules). Cada componente gestiona sus propias dependencias mediante el array `imports`.

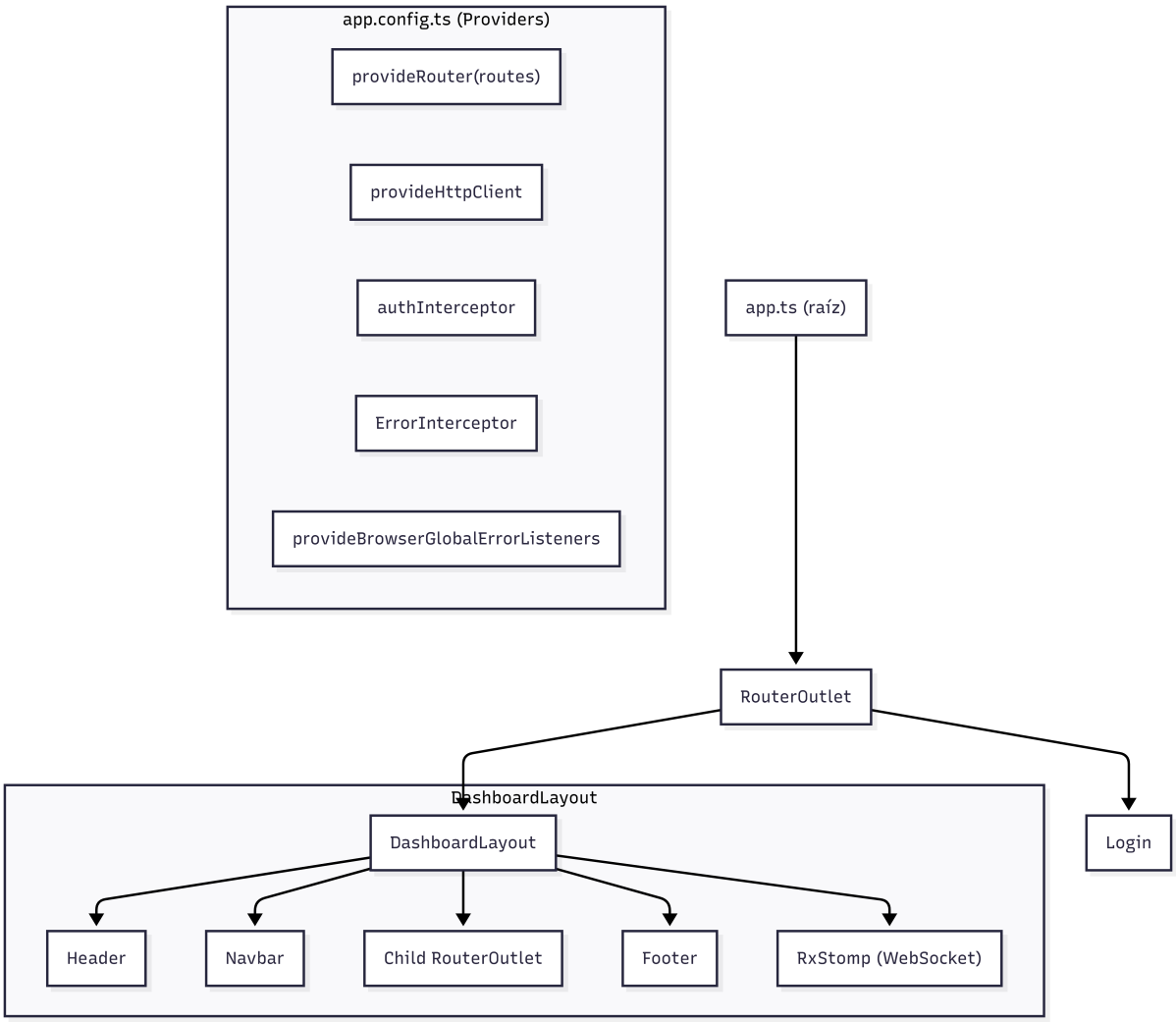

Figura 1: Arquitectura de componentes y proveedores

8. Rutas y navegación

Ruta	Componente	Guard(s)
`/`	`Login`	`loginGuard`
`/dashboard`	`DashboardLayout`	`authguardGuard`
`/dashboard` (home)	`DashboardComponent`	`tallerGuard`
`/dashboard/vehiculos`	`VehicleListComponent`	`tallerGuard`
`/dashboard/mantenimientos`	`WorkOrdersComponent`	`tallerGuard`
`/dashboard/mantenimientos/:id`	`WorkOrderDetailComponent`	`tallerGuard`
`/dashboard/taller`	`GestionTallerComponent`	`tallerGuard`
`/dashboard/perfil`	`GestionTallerComponent`	`tallerGuard`
`/dashboard/historial/:plate`	`VehicleHistoryComponent`	`tallerGuard`
`/dashboard/workShop`	`VisualizarTallerComponent`	`tallerGuard`
`/dashboard/alta-taller`	`AltaTaller`	`altaTallerGuard`
`**`	—	Redirige a `/dashboard`

9. Seguridad y autenticación

La autenticación se basa en **JWT gestionado mediante cookies seguras**.

9.1. Persistencia de sesión

Opción	Storage
Recuérdame: Sí	`localStorage`
Recuérdame: No	`sessionStorage`

9.2. Flujo de autenticación

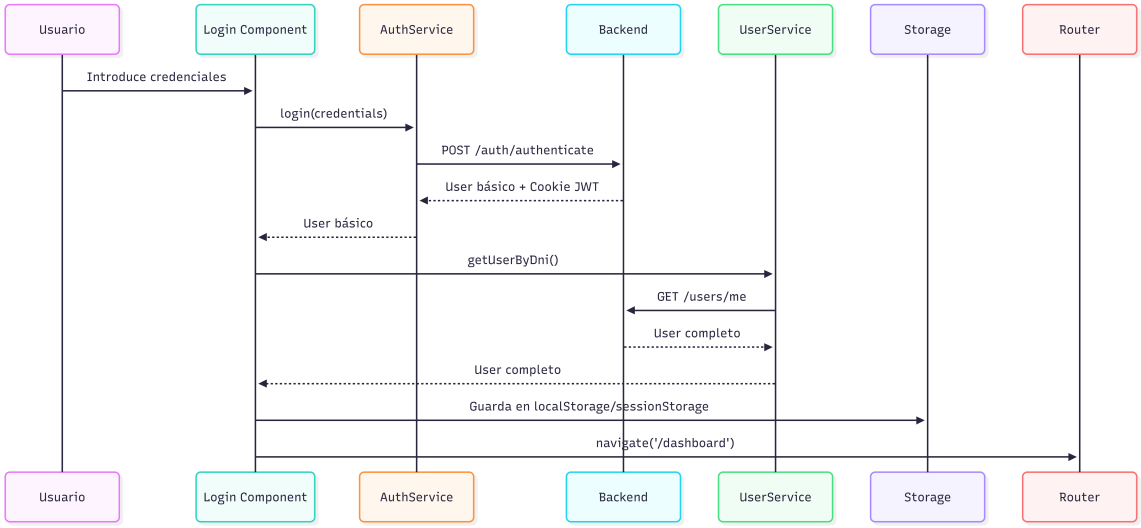

Figura 2: Flujo de autenticación

10. Interceptores HTTP

10.1. `authInterceptor`

- Solo actúa sobre peticiones dirigidas a `environment.apiUrl`
- Adjunta `withCredentials: true` para incluir la cookie JWT.
- Extrae e inyecta el token XSRF en la cabecera `X-XSRF-TOKEN`.

10.2. `ErrorInterceptor`

Código	Comportamiento
401	Limpia storage, redirige a `/`, muestra alerta SweetAlert2
403	Muestra alerta; si menciona "taller.º "sesión", limpia la sesión
429	Muestra alerta de "demasiados intentos"(rate limiting)
500	Muestra error genérico de servidor
0	Muestra error de "sin conexión con el servidor"

11. Funcionalidades por rol

Funcionalidad	MANAGER	CO_MANAGER	MECHANIC	CLIENT
Registrar taller	✓	×	×	×
Editar datos del taller	✓	✓	×	×
Invitar / contratar empleados	✓	✓	×	×
Despedir empleados	✓	✓	×	×
Ver flota completa del taller	✓	✓	×	×
Ver vehículos propios	✓	✓	✓	✓
Crear / editar / eliminar vehículo	✓	✓	✓	✓
Ver órdenes asignadas	×	×	✓	×
Crear orden de trabajo	✓	✓	×	×
Actualizar estado de orden	✓	✓	✓	×
Gestionar líneas de facturación	✓	✓	✓	×

12. Notificaciones en tiempo real

CarLog usa **WebSockets con protocolo STOMP** (`@stomp/rx-stomp`) para notificaciones instantáneas.

12.1. Tipos de notificación

Tipo	Efecto en el frontend
`FIRE`	Muestra alerta bloqueante y fuerza logout
`INVITE`	Muestra toast informativo + emite `NEW_INVITE`
`VEHICLE_REQUEST`	Muestra toast + emite `RELOAD_VEHICLES`

Tipo	Efecto en el frontend
`NEW_EMPLOYEE`	Emite `RELOAD_EMPLOYEES`
`NEW_FLEET_VEHICLE`	Emite `RELOAD_VEHICLES`

13. Conexión con el Backend

Dominio	Base URL	Descripción
Autenticación	`/auth`	Login, registro, logout
Usuarios	`/users`	Perfil, invitaciones, gestión de empleados
Vehículos	`/vehicles`	CRUD, flujo de ingreso/salida, historial
Órdenes	`/workorders`	CRUD de órdenes y líneas de facturación
Taller	`/workshop`	CRUD del taller y consulta de empleados
WebSocket	`/ws-carlog`	Canal STOMP para notificaciones

14. Licencia

Este proyecto es de uso académico/privado. Consulta con el autor para cualquier uso externo.